✦ JOSA ////////////////////

Issue Triage & Bug Reproduction

/* From a stranger's vague report to more tangible issues */

OpenLab Apprenticeship 2026

Week 03

// 1 ////////////////////

Learning Goals

- Reliably reproduce bugs in unfamiliar codebases on your own machine.
- Build **Minimal, Complete, Verifiable Examples (MCVEs)** that maintainers can act on immediately.
- Apply systematic debugging — bisection, logging, hypothesis-and-test — instead of "stare at the code harder."
- Triage a stale issue queue: confirm, duplicate, request info, or close.

// 2 ////////////////////

Technical Deep Dive

// 2.1 The MCVE: Minimum, Complete, Verifiable Example

A reproduction is **minimum** if removing any code stops it from reproducing the bug. **Complete** means it includes everything needed to run — no "and then I have a bunch of other config." **Verifiable** means anyone with the same environment can run it and see the same failure.

The discipline is brutal but mechanical. Start from your real failing code and:

1. Strip imports and dependencies you don't strictly need.
2. Inline functions you can; remove abstractions that aren't part of the bug.
3. Replace real data with the smallest synthetic data that still triggers the failure.
4. Reduce the call graph: can you trigger the bug with a single function call?
5. Re-run after every removal. The moment the bug stops reproducing, you've found a relevant piece — put it back.

A good MCVE for a JS bug looks like:

```
// node 20.11.0, on macOS 14.4
// Run: node repro.js
// Expected: prints "ok"
// Actual:   throws TypeError "Cannot read properties of undefined"
```

```
const { parse } = require("./lib/parser");

const input = '{"a":1,"b":'; // truncated input
console.log(parse(input));   // throws
```

Three lines of code, exact versions, expected vs. actual. A maintainer can paste it into their terminal and confirm in 30 seconds.

// 2.1 Bisection: the most underrated debugging technique

When a bug appeared "recently" but you don't know which commit introduced it, `git bisect` solves it in O(log n) commits:

```
git bisect start
git bisect bad                # current HEAD is broken
git bisect good v1.4.0        # this old version worked

# Git checks out the midpoint commit
# Run your test
git bisect bad   # or `good`
# Git narrows in. Repeat until it identifies the bad commit.
git bisect reset              # back to where you were
```

Even better: automate the test. If you can write a script that exits 0 when good and non-zero when bad:

```
git bisect run ./repro.sh
```

Git will run your script at each step and find the offending commit untouched by human hands. This is one of the highest-leverage tools in version control.

// 2.3 Print-debugging is fine, actually

There's a snobbery in some circles that "real engineers use a debugger." Ignore it. For most bug-reproduction work, well-placed `print` statements (or `console.log`, or `eprintln!`, or `log.Println`) are faster and more reliable than stepping through a debugger, especially in async code, web requests, or distributed systems where a breakpoint distorts timing.

That said, *do* learn one real debugger. For Python, the built-in `breakpoint()` is one keystroke away. For JavaScript, Chrome DevTools' Sources panel is genuinely good. For Go, `dlv debug`. For

Rust/C/C++, `lldb` or `gdb`. The 10% of bugs that print-debugging can't crack absolutely require these.

// 2.4 Triaging a stale issue queue

Many projects have hundreds of stale issues. Workflow for each:

1. **Can you reproduce it?** Yes → comment with confirmation, OS/version, and add a `confirmed` label if you have triage rights.
2. **Is it a duplicate?** Search. If yes, comment "Looks like a duplicate of #XYZ — closing in favor of that one" and link both ways.
3. **Is the report incomplete?** Ask politely: "Could you share the OS, version, and a minimal repro? I tried X and couldn't reproduce."
4. **Has it been fixed silently?** (Very common.) Try on `main`. If it works there, comment with the version and close. Or it could be a false positive or an environment problem from the issuer's side :)
5. **Is the project no longer interested?** Sometimes the maintainer has explicitly said "won't fix" or "out of scope." Close with a polite link to the relevant comment.

✓ **TIP**
The output of an hour of good triage is often more valuable to a maintainer than the same hour spent on code. Maintainers know this.

// 3 ////////////////////
Learning Material

Topic	Resource
The canonical MCVE guide	Stack Overflow — How to create a Minimal, Reproducible Example
Debugging mindset (and zines)	Julia Evans — jvns.ca and her Wizard Zines
Systematic debugging (book)	The Practice of Programming — Kernighan & Pike, Ch. 5
Git bisect tutorial	Pro Git — Debugging with Git
Project triage example	Rust — Triage Procedure facebook/react

Topic	Resource
	microsoft/vscode
Bug-report templates	Chromium — How to file a good bug

// 4 //////////////////

Suggested Repositories

- freeCodeCamp/freeCodeCamp — Visible triage labels.
- httpie/cli — `new` label for issues awaiting triage.
- microsoft/vscode — `info-needed` issues need a confirming reproduction.
- t3-oss/create-t3-app — `unconfirmed bug` is exactly this week's assignment.

// 5 //////////////////

Your Tasks for the Week

1. **Reproduce one issue on GitHub** in any of the suggested repos or other repos of choice. For each, post a comment with: your environment, the steps you ran, and the actual outcome. If it doesn't reproduce, say so.
2. **Build one MCVE.** Take an issue (could be the same one from task 1) or find an open issue with a sprawling reproduction and reduce it, then post the smaller reproduction as a comment. Or even better, open your own issue on a repo (not on your own repositories tho).
3. **Run a bisect.** On any project of your choice, find a behavior that changed (working in v1.0.0, broken in HEAD, or vice versa) and use `git bisect` to identify the exact commit. Document the command sequence and what you saw in your journal.
4. **Triage or help triaging an issue.** Submit a journal entry summarizing what you closed, duplicated, found as stale or false positive, or asked clarifying questions on.

// 6 //////////////////

Soft Skill of the Week: Active Listening (and Reading)

Issue triage is mostly *listening to a stranger try to describe a problem they don't fully understand themselves*. Active listening — the technique used by therapists, hostage negotiators, and good doctors — applies directly.

// 6.1 Three moves to practice

1. **Reflect to confirm.** Before responding, restate what you heard: "So the bug only happens when you're behind a corporate proxy and using Node 20+, right?" This is not patronizing; it's how you catch the 30% of cases where you misunderstood the problem.
2. **Ask for what's missing, not what's there.** People over-describe what they did and under-describe their environment. Default questions: OS, version, the minimum repro, what they expected vs. what happened.
3. **Don't interrupt with a solution.** A junior instinct is to read the first paragraph of an issue and immediately propose a fix. Resist. Read the full report. Read existing comments. Then propose.

The same skill applies to written threads. When a maintainer pushes back on your PR, the right move is almost always: read it twice, restate their concern in your reply ("If I understand right, you're worried that …"), and then respond. You'll be surprised how often the restatement alone resolves the disagreement.

// 6.2 Resources

- HBR — What Great Listeners Actually Do
- Julia Evans — How to ask good questions (the flip side of listening)